

HAMMERDOWN DETAILED CODE WALKTHROUGH

Equipment Management + Advanced History + Chat Tab + Nexus + Costs

Date: 2026-04-19

1) SCOPE AND MAIN FILES

Main implementation file:

- mainwindow.cpp

DB schema and static constraints:

- export_fixed.sql

Nexus tab core:

- nexuswidget.cpp
- nexuswidget_inference.cpp
- nexuswidget_ui.cpp

Costs tab core:

- costswidget.cpp
- costswidget_engine.cpp

2) EQUIPMENT MODULE ARCHITECTURE (APP SIDE)

2.1 Equipment page + extra tabs creation

- mainwindow.cpp:1349-1359
 - Equipment page is created.
 - NEXUS tab is injected programmatically:
ui_equipment->tabWidget->addTab(m_nexusWidget, "NEXUS");
 - COSTS tab is injected programmatically:
ui_equipment->tabWidget->addTab(m_costsWidget, "COSTS");

2.2 Equipment CRUD button wiring

- mainwindow.cpp:1775-1781
 - Add button -> onEquipmentAdd
 - Modify button -> onEquipmentModify
 - Delete button -> onEquipmentDelete
 - Delete confirm button -> onEquipmentDelete
 - Search button and Enter in search input -> onEquipmentSearch

2.3 History and bulk operations wiring

- mainwindow.cpp:1844-1847, 7388-7395
 - Refresh/Search/Clear/Export history actions are connected.
 - Bulk status update and bulk delete all are connected.

2.4 Chat tab wiring inside equipment module

- mainwindow.cpp:1386-1389
 - Send button + Enter key -> onChatSendMessage
 - Employee list click -> onChatEmployeeSelected
 - Attach image button -> onChatAttachImage
- mainwindow.cpp:1833, 7411, 12245
 - chatRefreshTimer starts every 3000 ms on chat-related events.

2.5 Equipment tab mode setup function

Function head:

```
- mainwindow.cpp:7199
  void MainWindow::setupEquipmentModes()
```

What it does:

- Builds Add mode vs Manage mode radio behavior.
- Shows/hides ID and action buttons depending on mode.
- Builds and updates form progress bar widgets.
- Connects progress signals to updateEquipProgress.
- Wires history and bulk controls in same setup region.

3) DATABASE FOUNDATIONS FOR EQUIPMENT

3.1 Equipment table and constraints (static SQL)

```
- export_fixed.sql:200
  CREATE TABLE EQUIPMENT (...)
- export_fixed.sql:385
  PK_EQUIPMENT primary key on EQUIPMENT_ID
- export_fixed.sql:388
  CHK_EQUIPMENT_STATUS check constraint
- export_fixed.sql:428
  FK_EQUIPMENT_EMPLOYEE foreign key to EMPLOYEES
```

3.2 ID generation trigger (static SQL)

```
- export_fixed.sql:306
  CREATE OR REPLACE TRIGGER TRG_EQUIPMENT_BI
- export_fixed.sql:311
  SELECT SEQ_EQUIPMENT.NEXTVAL INTO :NEW.EQUIPMENT_ID
```

3.3 Runtime bootstrap for history objects (dynamic SQL in app)

Function head:

```
- mainwindow.cpp:13760
  void MainWindow::ensureEquipmentHistoryDatabaseObjects()
```

Inside this function:

- Creates EQUIPMENT_HISTORY table (if not already existing).
- Creates EQUIPMENT_HISTORY_SEQ sequence (if missing).
- Creates index IDX_EQUIPMENT_HISTORY_OP_DATE.
- Creates trigger TRG_EQUIPMENT_HISTORY_AUDIT on EQUIPMENT.

Trigger logic written by app:

- On INSERT: writes ADD row to EQUIPMENT_HISTORY.
- On UPDATE: writes MODIFY row to EQUIPMENT_HISTORY.
- On DELETE: writes DELETE row to EQUIPMENT_HISTORY.

This is the key to direct-SQL history capture (outside app UI).

4) EQUIPMENT CRUD - DETAILED

4.1 Refresh/View listing

Function head:

```
- mainwindow.cpp:10305
void MainWindow::onEquipmentRefreshView()
```

Core SQL:

```
- SELECT ... FROM EQUIPMENT WHERE STATUS != 'Retired' ORDER BY
EQUIPMENT_ID
```

Behavior:

- Builds table model with two action columns:
Action, Delete, then ID/Type/Qty/Price/Status/Purchase
Date/Description.
- Sets row selection and read-only behavior.

4.2 Add equipment

Function head:

```
- mainwindow.cpp:10320
void MainWindow::onEquipmentAdd()
```

Validation:

- Blocks add if ID field is already populated (record already loaded).
- Requires non-empty type + description.
- Requires quantity > 0 and unit price > 0.

Core SQL:

```
- INSERT INTO EQUIPMENT (...) VALUES ((SELECT NVL(MAX(EQUIPMENT_ID), 0) +
1 FROM EQUIPMENT), ...)
```

After successful insert:

- Retrieves max id for metadata/logging.
- Calls playEquipSuccessAnimation.
- Calls logActivity with equipment_operation = ADD and rich metadata.
- Refreshes form, table, history, progress.
- Reinitializes Nexus widget.

4.3 Modify equipment

Function head:

```
- mainwindow.cpp:10397
void MainWindow::onEquipmentModify()
```

Flow:

- Requires ID.
- Verifies DB connection is open.
- Loads existing row first:
SELECT ... FROM EQUIPMENT WHERE EQUIPMENT_ID = :id
- Uses existing DB values as fallback for empty/invalid form values.
- Prevents empty final type.

Core SQL update:

```
- UPDATE EQUIPMENT SET ... , NEXT_MAINTENANCE = SYSDATE WHERE
EQUIPMENT_ID = :id
```

After successful update:

- Shows success message.
- Calls logActivity with equipment_operation = MODIFY.

- Refreshes form/table/history.
- Reinitializes Nexus widget.

4.4 Delete equipment

Function head:

- mainwindow.cpp:10488
void MainWindow::onEquipmentDelete()

Flow:

- Gets target either from selected row or form fields.
- Confirms deletion via QMessageBox.

Core SQL:

- DELETE FROM EQUIPMENT WHERE EQUIPMENT_ID = :id

Behavior:

- Hard delete (row removed from EQUIPMENT table).
- Calls logActivity with equipment_operation = DELETE.
- Refreshes form/table/history.
- Reinitializes Nexus widget.

4.5 Search equipment

Function head:

- mainwindow.cpp:10556
void MainWindow::onEquipmentSearch()

Core SQL pattern:

- Base: FROM EQUIPMENT WHERE STATUS != 'Retired'
- Optional text filter over description/type/id
- Optional status filter from cb_filter_status

Result:

- Rebinds table model with filtered data.

4.6 Clear form fields helper

Function head:

- mainwindow.cpp:5808
void MainWindow::onEquipmentClearFields()

What resets:

- ID, type, date, description, quantity, price, status.

4.7 Progress and success UX helpers

Function heads:

- mainwindow.cpp:11155
void MainWindow::updateEquipProgress()
- mainwindow.cpp:11284
void MainWindow::playEquipSuccessAnimation(const QString &equipName)

What they do:

- updateEquipProgress: computes 0/25/50/75/100 based on type/date/price/desc completeness.
- playEquipSuccessAnimation: plays sound + visual highlight + flying card/confetti effect.

4.8 Bulk advanced operations

Function heads:

- mainwindow.cpp:12926
void MainWindow::onEquipmentBulkUpdateStatus()
- mainwindow.cpp:12979
void MainWindow::onEquipmentDeleteAll()

Bulk update:

- Iterates selected rows.
- SQL per row: UPDATE EQUIPMENT SET STATUS = :status WHERE EQUIPMENT_ID = :id

Delete all:

- Confirms destructive action.
- Executes DELETE FROM EQUIPMENT.
- Logs mass deletion.
- Refreshes table/history/stats + Nexus + Costs.

4.9 Export functions

Function heads:

- mainwindow.cpp:11412
void MainWindow::onEquipmentExportPDF()
- mainwindow.cpp:11507
void MainWindow::onEquipmentExportStatsPDF()

Behavior:

- Exports equipment registry PDF and analytics PDF through QPainter/QPrinter.

5) ADVANCED HISTORY - HOW IT REALLY WORKS

This project uses a dual-path history strategy:

Path A (preferred): Oracle table EQUIPMENT_HISTORY via trigger.

Path B (fallback): JSON file hammerdown_audit_log.json.

5.1 History refresh/search entry points

Function heads:

- mainwindow.cpp:10750
void MainWindow::onEquipmentHistoryRefresh()
- mainwindow.cpp:10755
void MainWindow::onEquipmentHistorySearch()

5.2 DB-first history loading

Inside onEquipmentHistorySearch:

- Builds SQL against EQUIPMENT_HISTORY with operation filter.
- Loads 3 separate views:
 - ADD history table
 - MODIFY history table
 - DELETE history table

The DB SQL starts from:

- FROM EQUIPMENT_HISTORY WHERE UPPER(OPERATION_TYPE) = :op
and supports optional search predicates.

5.3 JSON fallback history loading

If DB queries fail:

- Reads hammerdown_audit_log.json.
- Parses Equipment module records.
- Uses regex to infer operation from action_details when explicit fields are missing:
 - Added new equipment: ... (ID: N)
 - Modified equipment: ... (ID: N)
 - Deleted equipment: ... (ID: N)
- Backfills type/description/status/price/date columns.

5.4 History clear policy

Function head:

- mainwindow.cpp:10953
void MainWindow::onEquipmentHistoryClear()

Behavior:

- No physical clear of history.
- Shows info: history is logically linked and cannot be cleared separately.

5.5 Context menus linked to Nexus

Function heads:

- mainwindow.cpp:10958
void MainWindow::onEquipmentCustomContextMenu(const QPoint &pos)
- mainwindow.cpp:10978
void MainWindow::onEquipmentHistoryCustomContextMenu(const QPoint &pos, int tableIdx)

Behavior:

- Equipment view menu can open selected equipment in Nexus graph analysis.
- History tables menu can open Nexus time machine at selected row date.

5.6 JSON audit writer used by CRUD

Function head:

- mainwindow.cpp:16301
void MainWindow::logActivity(const QString &action, const QString &module, const QJsonObject &extra)

What it writes to hammerdown_audit_log.json:

- log_id
- timestamp_iso and timestamp_ms
- action_details, module_name
- employee_name
- any extra metadata (equipment_operation, equipment_id, etc.)

5.7 End-to-end add/modify/delete path (app + DB + visible result)

A) User clicks Add/Modify/Delete in app:

- App writes to EQUIPMENT table.
- DB trigger TRG_EQUIPMENT_HISTORY_AUDIT writes to EQUIPMENT_HISTORY.

- App also writes rich audit JSON using `logActivity`.
- UI refreshes equipment table and history tables.

B) User (or admin) edits EQUIPMENT directly in SQL tools:

- Trigger still writes to `EQUIPMENT_HISTORY`.
- `onEquipmentHistorySearch` shows those changes via DB path.
- JSON may not contain those edits (expected).

=====

6) CHAT TAB - FULL FUNCTIONALITY EXPLANATION

=====

6.1 Core chat function heads and lines

- `mainwindow.cpp:11702` `onChatEnsureTable`
- `mainwindow.cpp:11709` `onChatSendMessage`
- `mainwindow.cpp:11776` `onChatRefresh`
- `mainwindow.cpp:12144` `onChatEmployeeListRefresh`
- `mainwindow.cpp:12170` `onChatAttachImage`
- `mainwindow.cpp:12193` `onChatEmployeeSelected`
- `mainwindow.cpp:12254` `onChatDeleteMessage`
- `mainwindow.cpp:12295` `onChatEmojiClicked`
- `mainwindow.cpp:12396` `onChatGifClicked`
- `mainwindow.cpp:12618` `onChatSettingsClicked`
- `mainwindow.cpp:13068` `showUnreadMessagesSplash`
- `mainwindow.cpp:13276` `onChatSearchToggle`
- `mainwindow.cpp:13513` `onChatVoiceToggled`
- `mainwindow.cpp:13523` `onChatStartRecord`
- `mainwindow.cpp:13533` `onChatStopRecord`
- `mainwindow.cpp:13500` `shakeWidget`

6.2 Data storage model

Primary chat file:

- `hammerdown_chat.json`

Message object fields include:

- `sender_id`
- `receiver_id`
- `message`
- `timestamp`
- optional `image_data` (base64)
- optional `gif_url` and `gif_title`

Deleted messages backup:

- `hammerdown_chat_trash.json`

6.3 Send message logic

`onChatSendMessage`:

- Requires selected chat partner.
- Requires non-empty text or attached image.
- Saves frequent text to completer list (autocomplete support).
- Appends message object to JSON file.
- Clears input and attachment preview.
- Calls `onChatRefresh`.

6.4 Render and refresh logic

onChatRefresh:

- Loads JSON file.
- Skips repaint if no count/partner change (anti-flicker optimization).
- Rebuilds chat bubble widgets for current conversation.
- Supports:
 - text bubbles
 - image bubbles
 - GIF bubbles (download + QMovie animation)
 - voice-note style message rendering
- Adds delete button for own messages.
- Formats human-readable timestamps.
- Maintains scroll-to-bottom behavior.
- Updates tab badge when new incoming message appears off-tab.

6.5 Employee list and partner selection

onChatEmployeeListRefresh:

- Queries EMPLOYEES and excludes current user.
- Populates list with name + ID.

onChatEmployeeSelected:

- Sets currentChatPartnerId.
- Updates chat header labels/avatar initials.
- Computes and shows last-seen status from chat JSON.
- Starts refresh timer and refreshes conversation.

6.6 Attachments and rich inputs

onChatAttachImage:

- Opens file picker.
- Reads binary image and stores in pendingChatImage.
- Shows UI preview label.

onChatEmojiClicked:

- Popup emoji picker with categories.
- Inserts selected emoji into chat input.

onChatGifClicked:

- Popup GIF finder (GIPHY API).
- Shows trending/search GIFs.
- Saves selected GIF as message object with gif_url/gif_title.

6.7 Message deletion and restore

onChatDeleteMessage:

- Moves selected message to hammerdown_chat_trash.json with deleted_at timestamp.
- Removes it from main chat file.

onChatSettingsClicked:

- Provides clear conversation action (pair-specific wipe).
- Provides restore deleted messages UI from trash file.
- Includes ambient volume control for chat music.

6.8 Search and voice

onChatSearchToggle:

- Creates/toggles in-tab search bar.

- Highlights matched text using rich text spans.
- Supports next/previous navigation between matches.

onChatVoiceToggled/onChatStartRecord/onChatStopRecord:

- Records audio note to local voice_notes directory.
- Stop action auto-sends a "Voice Note" placeholder message.

shakeWidget:

- Small shake animation used when user tries to send empty content.

6.9 Unread inbox-style splash

showUnreadMessagesSplash:

- Counts unread messages by comparing incoming timestamps vs last sent per partner.
- Displays animated popup after login.
- Can navigate directly to chat tab and auto-select first unread sender.

=====

7) SHORT NEXUS TAB EXPLANATION

=====

7.1 Key function heads

- nexuswidget.cpp:23
QList<NexusEquipment> loadAllEquipment()
- nexuswidget_inference.cpp:14
void InferenceEngine::loadData()
- nexuswidget_inference.cpp:42
void InferenceEngine::runAllRules()
- nexuswidget_ui.cpp:3245
void NexusWidget::runAiReport()
- nexuswidget_ui.cpp:3270
void NexusWidget::runAiInference()
- nexuswidget_ui.cpp:3311
void NexusWidget::runAiDecision(const QString &action, int equipId)
- nexuswidget_ui.cpp:3330
void NexusWidget::switchTab(int index)
- nexuswidget_ui.cpp:3340
void NexusWidget::initialize()

7.2 How it works in short

- loadAllEquipment reads equipment rows from DB (excluding Retired).
- InferenceEngine loads equipment/employees and executes rule-based insight generation.
- NexusWidget has internal sub-tabs (Graph, Time, Report, Decisions, Maintenance, AI Forge, Futures Lab).
- runAiReport/runAiInference/runAiDecision call Groq/Gemini compatible endpoint wrapper and update UI widgets.
- initialize handles first-load animation then data/rule loading; later calls do refresh-only.

7.3 Connection with equipment CRUD/history

- After add/modify/delete, MainWindow calls m_nexusWidget->initialize() to refresh analytics.
- Equipment context menu can jump to Nexus graph with selected equipment.
- History context menu can jump to Nexus time machine for selected date.

```
=====
8) SHORT COSTS TAB EXPLANATION
=====
```

```
8.1 Key function heads
```

```
- costswidget_engine.cpp:13    void CostCalculationEngine::calculate()
- costswidget.cpp:99           void CostsWidget::initialize()
- costswidget.cpp:122          void CostsWidget::onDataReady()
- costswidget.cpp:162          void CostsWidget::switchSubTab(int index)
- costswidget.cpp:172          void CostsWidget::requestGroqInsight()
```

```
8.2 How it works in short
```

```
- CostCalculationEngine::calculate loads EQUIPMENT rows and derives
financial metrics:
    TCO, maintenance/operational estimates, depreciation, ROI score,
recommendation,
    repair vs replace values, monthly replacement budget, urgent flags.
- CostsWidget::initialize runs loading animation then triggers
calculations.
- onDataReady pushes computed dataset into each Costs sub-widget:
    ownership, repair/replace, forecast, ROI, timeline, report.
- switchSubTab changes the visible sub-widget and re-requests AI insight.
- requestGroqInsight builds tab-specific financial prompts and calls
CostGroqClient.
```

```
=====
9) COMPLETE FUNCTION HEAD INDEX (EQUIPMENT + CHAT + HISTORY)
=====
```

```
Equipment core:
```

```
- mainwindow.cpp:5808    void MainWindow::onEquipmentClearFields()
- mainwindow.cpp:6033    void MainWindow::setupEquipmentStats()
- mainwindow.cpp:7199    void MainWindow::setupEquipmentModes()
- mainwindow.cpp:10305   void MainWindow::onEquipmentRefreshView()
- mainwindow.cpp:10320   void MainWindow::onEquipmentAdd()
- mainwindow.cpp:10397   void MainWindow::onEquipmentModify()
- mainwindow.cpp:10488   void MainWindow::onEquipmentDelete()
- mainwindow.cpp:10556   void MainWindow::onEquipmentSearch()
- mainwindow.cpp:10750   void MainWindow::onEquipmentHistoryRefresh()
- mainwindow.cpp:10755   void MainWindow::onEquipmentHistorySearch()
- mainwindow.cpp:10953   void MainWindow::onEquipmentHistoryClear()
- mainwindow.cpp:10958   void
MainWindow::onEquipmentCustomContextMenu(const QPoint &pos)
- mainwindow.cpp:10978   void
MainWindow::onEquipmentHistoryCustomContextMenu(const QPoint &pos, int
tableIdx)
- mainwindow.cpp:11155   void MainWindow::updateEquipProgress()
- mainwindow.cpp:11284   void MainWindow::playEquipSuccessAnimation(const
QString &equipName)
- mainwindow.cpp:11412   void MainWindow::onEquipmentExportPDF()
```

```

- mainwindow.cpp:11507 void MainWindow::onEquipmentExportStatsPDF()
- mainwindow.cpp:12926 void MainWindow::onEquipmentBulkUpdateStatus()
- mainwindow.cpp:12979 void MainWindow::onEquipmentDeleteAll()
- mainwindow.cpp:13036 void MainWindow::onEquipmentShareToChat()
- mainwindow.cpp:13760 void
MainWindow::ensureEquipmentHistoryDatabaseObjects()
- mainwindow.cpp:16301 void MainWindow::logActivity(const QString
&action, const QString &module, const QJsonObject &extra)

```

Chat tab:

```

- mainwindow.cpp:11702 void MainWindow::onChatEnsureTable()
- mainwindow.cpp:11709 void MainWindow::onChatSendMessage()
- mainwindow.cpp:11776 void MainWindow::onChatRefresh()
- mainwindow.cpp:12144 void MainWindow::onChatEmployeeListRefresh()
- mainwindow.cpp:12170 void MainWindow::onChatAttachImage()
- mainwindow.cpp:12193 void
MainWindow::onChatEmployeeSelected(QListWidgetItem *item)
- mainwindow.cpp:12254 void MainWindow::onChatDeleteMessage(int index)
- mainwindow.cpp:12295 void MainWindow::onChatEmojiClicked()
- mainwindow.cpp:12396 void MainWindow::onChatGifClicked()
- mainwindow.cpp:12618 void MainWindow::onChatSettingsClicked()
- mainwindow.cpp:13068 void MainWindow::showUnreadMessagesSplash()
- mainwindow.cpp:13276 void MainWindow::onChatSearchToggle()
- mainwindow.cpp:13500 void MainWindow::shakeWidget(QWidget *w)
- mainwindow.cpp:13513 void MainWindow::onChatVoiceToggled()
- mainwindow.cpp:13523 void MainWindow::onChatStartRecord()
- mainwindow.cpp:13533 void MainWindow::onChatStopRecord()
- mainwindow.cpp:12818 void MainWindow::setupChatForgeVisuals()
- mainwindow.cpp:12823 void MainWindow::enforceChatTabTopOffset()
- mainwindow.cpp:12828 void MainWindow::onWeatherAssistantClicked()

```

Nexus short index:

```

- nexuswidget.cpp:23 QList<NexusEquipment> loadAllEquipment()
- nexuswidget_inference.cpp:14 void InferenceEngine::loadData()
- nexuswidget_inference.cpp:42 void InferenceEngine::runAllRules()
- nexuswidget_ui.cpp:3245 void NexusWidget::runAiReport()
- nexuswidget_ui.cpp:3270 void NexusWidget::runAiInference()
- nexuswidget_ui.cpp:3311 void NexusWidget::runAiDecision(const
QString &action, int equipId)
- nexuswidget_ui.cpp:3330 void NexusWidget::switchTab(int index)
- nexuswidget_ui.cpp:3340 void NexusWidget::initialize()

```

Costs short index:

```

- costswidget_engine.cpp:13 void CostCalculationEngine::calculate()
- costswidget.cpp:99 void CostsWidget::initialize()
- costswidget.cpp:122 void CostsWidget::onDataReady()
- costswidget.cpp:162 void CostsWidget::switchSubTab(int index)
- costswidget.cpp:172 void CostsWidget::requestGroqInsight()

```

=====

10) PRACTICAL ANSWER TO YOUR KEY QUESTION

=====

"How add/modify/delete from app and from DB appears in history?"

- From app buttons:
 - 1) CRUD writes EQUIPMENT row changes.
 - 2) DB trigger writes EQUIPMENT_HISTORY row (ADD/MODIFY/DELETE).
 - 3) App also writes JSON audit entry through logActivity.
 - 4) History tab displays DB data first; if unavailable, it falls back to JSON.

- From direct SQL outside app:
 - 1) INSERT/UPDATE/DELETE on EQUIPMENT still fires TRG_EQUIPMENT_HISTORY_AUDIT.
 - 2) History tab reads that from EQUIPMENT_HISTORY.
 - 3) JSON logActivity is not required for this DB path.

End of report.